

Aufgabe 3: Adressierungsbeispiel

Florian Breitwieser
& Jakob Zimmermann

5. Mai 2026

1 Einleitung

Dieses Dokument beschreibt die Umsetzung des Adressierungsbeispiels als Teil der Aufgabe 3. Es fasst die Architektur des Address-Slave und des Address-Master zusammen, dokumentiert die Simulation und beschreibt Metastabilitätseffekte.

2 Aufgabenstellung

Ziel ist die Implementierung eines adressierbaren LED-Slave-Moduls für das iceduino-Board und eines synthetisierbaren Masters, der die LEDs in einem Lauflicht ansteuert. Die Abgabeanforderungen umfassen:

- Slave Code und erläuternder Text zur Funktion
- Slave Testbench Code und erläuternder Text zur Funktion, inklusive asserts und Hinweis auf nicht-synthetisierbaren Testbench-Code
- Simulation Slave mit Testbench (Screenshot + Analyse)
- Master Blockschaltbild inkl. Slave-Anbindung
- Master Code
- Master Simulation mit Slave
- Erläuterung der Simulationsergebnisse und Master-Statemachine
- Metastabilitätsbetrachtung: Ursache, Auswirkung und Vermeidung

3 Addr_Slave Code

Der folgende Code zeigt das Low-Level-Verhalten des Slave-Moduls. Es verarbeitet den Reset, liest die Adresse und schreibt bei gesetztem `we` das Datensignal `d` in das entsprechende LED-Register.

```
1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity addr_slave is
6      port(
7          clk: in std_logic;
8          we : in std_logic;
9          d  : in std_logic;
10         rst : in std_logic;
11         addr : in std_logic_vector(2 downto 0);
12         leds : out std_logic_vector(7 downto 0)
13     );
14 end entity;
15
16 architecture addr_slave_a of addr_slave is
17 begin
18     process(clk, rst) is
19     begin
20         if rst = '1' then
21             leds <= (others => '0');
22         elsif rising_edge(clk) then
23             if we = '1' then
24                 leds(to_integer(unsigned(addr))) <= d;
25             end if;
26         end if;
27     end process;
28 end architecture addr_slave_a;
```

Bei steigender Flanke des Taktsignals `clk` und aktivem Reset (`rst = '1'`) werden alle LEDs gelöscht. Wenn `we` gesetzt ist, wird das Datenbit `d` an der Position geschrieben, die durch die Adresse `addr` bestimmt wird. Die LEDs werden als 8-Bit-Vektor ausgegeben, wobei jedes Bit eine LED repräsentiert.

4 Addr_Slave Testbench

Die Testbench prüft die korrekte Adressierung des Slave-Moduls für mehrere Adressen und enthält Assert-Anweisungen für die LED-Ausgabe.

```
1  library ieee;
2  use IEEE.std_logic_1164.all;
3  use IEEE.numeric_std.all;
4
5  entity addr_slave_tb is
6  end entity;
7
8  architecture addr_slave_tb_a of addr_slave_tb is
9      signal clk : std_logic := '0';
10     signal we : std_logic := '0';
11     signal d : std_logic := '0';
12     signal reset : std_logic := '0';
13     signal addr : std_logic_vector(2 downto 0) := (others => '0');
14     signal leds : std_logic_vector(7 downto 0) := (others => '0');
15 begin
16     clk <= not clk after 10 ns; -- 50 MHz
17
18     addr_slave_instance : entity work.addr_slave
19         port map(
20             clk => clk,
21             we => we,
22             d => d,
23             reset => reset,
24             addr => addr,
25             leds => leds
26         );
27
28     process is
29     begin
30         reset <= '1';
31         wait for 30 ns;
32         reset <= '0';
33
34         addr <= (others => '0');
35         wait for 10 ns;
36         d <= '1';
37         we <= '1';
38
39         wait for 80 ns;
40         assert (leds = "00000001") report "R1: LEDS falsch" severity error;
41         assert (addr = "000") report "R1: b0 falsch" severity error;
42         report "R1: OK" severity note;
43         we <= '0';
44         wait for 1 ns;
45         d <= '0';
```

```

46
47     wait for 200 ns;
48     addr <= (others => '1');
49     wait for 10 ns;
50     d <= '1';
51     we <= '1';
52
53     wait for 80 ns;
54     assert (leds = "10000001") report "R2: LEDS falsch" severity error;
55     assert (addr = "111") report "R2: addr falsch" severity error;
56     report "R2: OK" severity note;
57     we <= '0';
58     wait for 1 ns;
59     d <= '0';
60
61     wait for 200 ns;
62     addr <= "010";
63     wait for 10 ns;
64     d <= '1';
65     we <= '1';
66
67     wait for 80 ns;
68     assert (leds = "10000101") report "R3: LEDS falsch" severity error;
69     report "R3: OK" severity note;
70     we <= '0';
71     wait for 200 ns;
72     d <= '0';
73     end process;
74 end architecture addr_slave_tb_a;

```

Alle Asserts konnten erfolgreich durchlaufen werden. Der addr-slave funktioniert einwandfrei.

Alle 'wait' und 'after' Anweisungen sind nicht-synthetisierbar und dienen ausschließlich der Simulation.

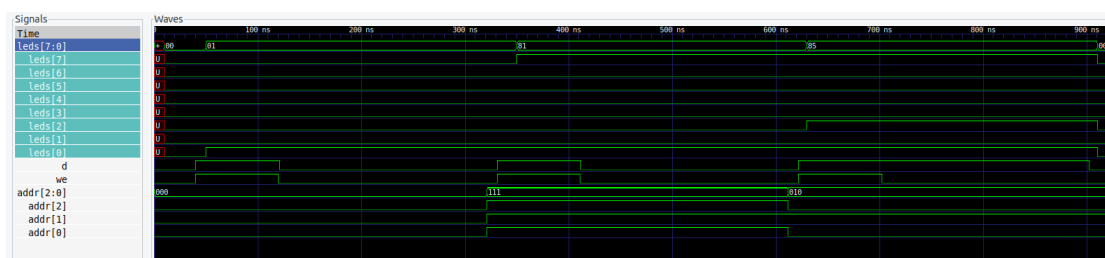


Abbildung 1: Simulation des Address-Slave mit Testbench

Die Simulation zeigt die korrekte Adressierung und LED-Ausgabe für die getesteten Adressen. Bei Adresse '000' leuchtet die erste LED, bei '111' die letzte LED, und bei '010' die dritte LED. Alle Asserts bestätigen die erwarteten Ergebnisse.

5 Master Architektur und Code

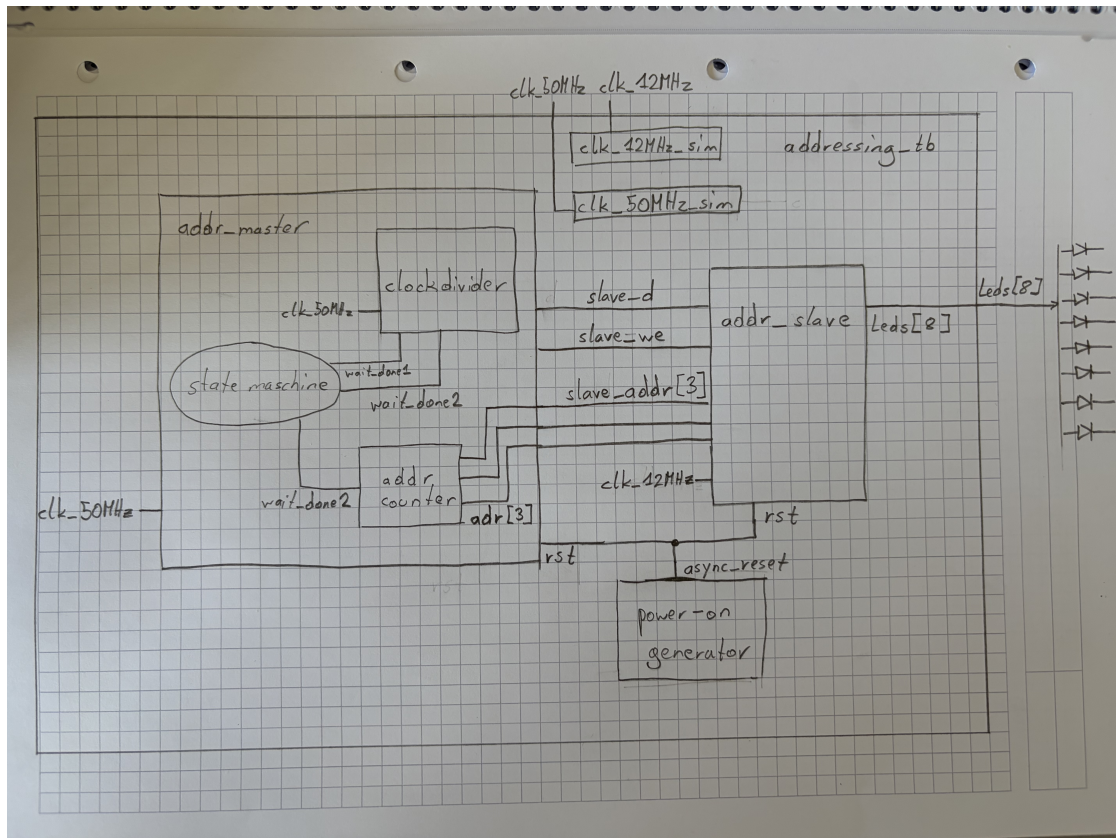


Abbildung 2: Blockschaltbild des Address-Master und Slave-Anbindung

Das Bild zeigt die Zusammenschaltung des Address-Master mit dem Address-Slave. Der Master generiert die Signale **adr**, **we** und **d**, die direkt an den Slave weitergegeben werden. Ein Power-On-Reset-Generator sorgt für einen definierten Reset-Zustand nach FPGA-Neustart.

Der **addr_master** erzeugt die Steuerungsimpulse für den Slave. Er durchläuft die Adressen 000 bis 111 und schaltet jeweils eine LED ein, wartet dann eine definierte Verzögerungszeit, schaltet ab und geht zur nächsten Adresse.

```

1 entity addr_master is
2   generic (
3     variable_delay : integer := 12500000
4   );
5   port(
6     clk : in std_logic;
7     rst : in std_logic;
8     adr : out std_logic_vector(2 downto 0);

```

```

9         we : out std_logic;
10        d : out std_logic
11    );
12 end entity;
13
14 architecture addr_master_a of addr_master is
15     type state_type is (START, LON, LOFF);
16     signal state : state_type := START;
17     signal cnt : integer range 0 to 12500000 := 0;
18     signal adr_reg : unsigned(2 downto 0) := "000";
19     signal delay_cnt : integer range 0 to variable_delay := 0;
20     signal slave_we : std_logic := '0';
21     signal slave_d : std_logic := '0';
22     signal delay_cnt2 : integer range 0 to 20000 := 0;
23     signal wait_done1 : std_logic := '0';
24     signal wait_done2 : std_logic := '0';
25 begin
26
27     process (clk, rst)
28     begin
29
30         if rst = '1' then
31             state <= START;
32             slave_we <= '0';
33             slave_d <= '0';
34             wait_done1 <= '0';
35             wait_done2 <= '0';
36         elsif rising_edge(clk) then
37             case state is
38                 when START =>
39                     state <= LON;
40                     wait_done1 <= '0';
41                     wait_done2 <= '0';
42                 when LON =>
43                     slave_d <= '1';
44                     if delay_cnt = 0 then
45                         slave_we <= '1';
46                         delay_cnt <= delay_cnt + 1;
47                     elsif delay_cnt = 10 then
48                         slave_we <= '0';
49                         delay_cnt <= delay_cnt + 1;
50                     elsif delay_cnt = variable_delay then
51                         delay_cnt <= 0;
52                         wait_done1 <= '1';
53                     else
54                         delay_cnt <= delay_cnt + 1;
55                     end if;
56
57                     if wait_done1 = '1' then
58                         state <= LOFF;

```

```

59         wait_done1 <= '0';
60     end if;
61     when LOFF =>
62         slave_d <= '0';
63         slave_we <= '1';
64
65         if delay_cnt2 = 10 then
66             delay_cnt2 <= 0;
67             slave_we <= '0';
68             wait_done2 <= '1';
69         else
70             delay_cnt2 <= delay_cnt2 + 1;
71         end if;
72
73         if wait_done2 = '1' then
74             if adr_reg = "111" then
75                 adr_reg <= "000";
76             else
77                 adr_reg <= adr_reg + 1;
78             end if;
79
80             state <= LON;
81             wait_done2 <= '0';
82         end if;
83     end case;
84 end if;
85 end process;
86
87 adr <= std_logic_vector(adr_reg);
88 d <= slave_d;
89 we <= slave_we;
90 end architecture addr_master_a;

```

Der Master verwendet einen Zustandsautomaten mit den Zuständen:

- **START**
- **LON** (LED einschalten)
- **LOFF** (LED ausschalten)

Das folgende Zustandsdiagramm zeigt die Übergänge zwischen den Zuständen.

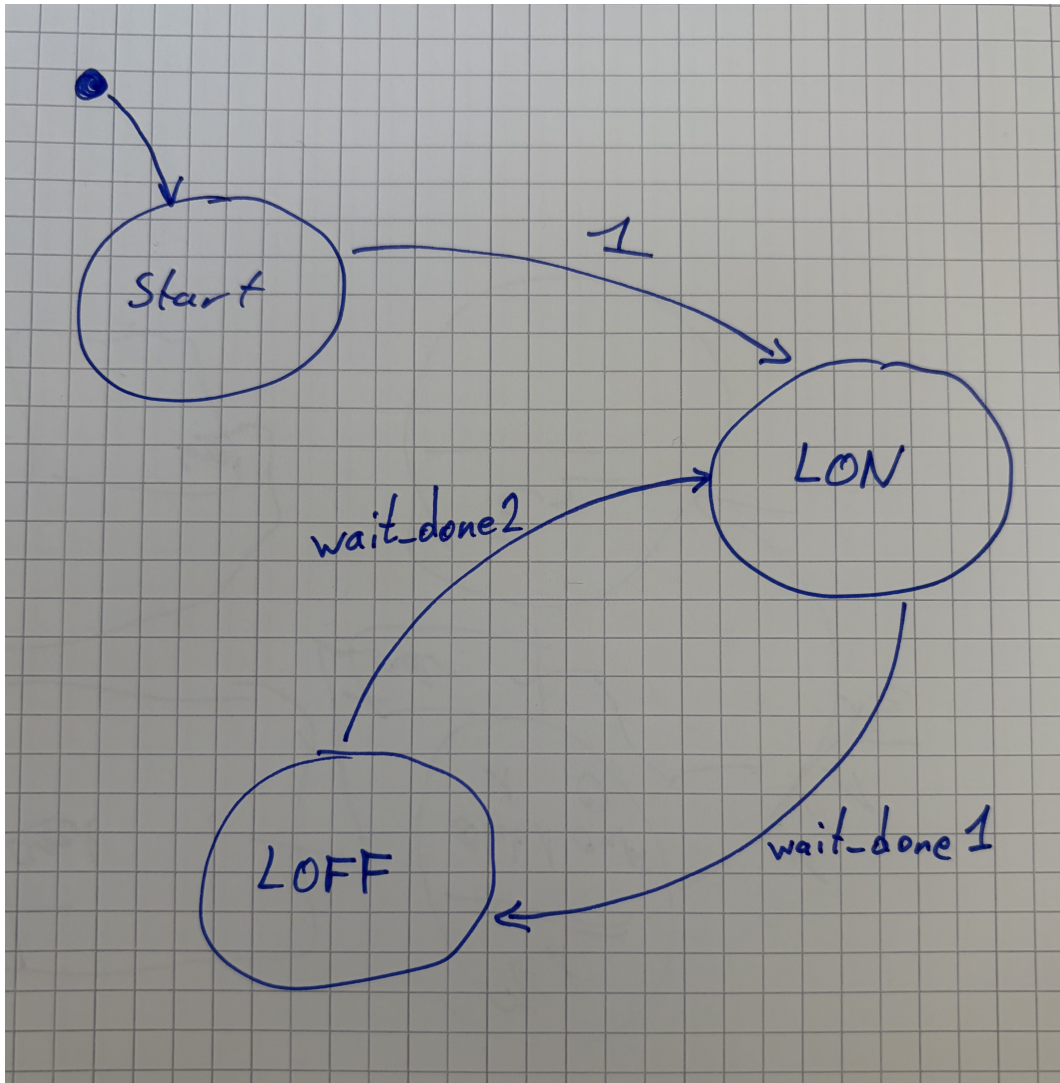


Abbildung 3: Zustandsdiagramm des Address-Master

6 Testbench

Die Testbench `addressing_tb.vhd` implementiert die Addressing-Simulation mit verkürztem Delay.

```
1  entity addressing_tb is
2      generic (
3          variable_delay : integer := 500
4      );
5
6      port(
7          clk_12MHz : in std_logic;
8          clk_50MHz : in std_logic;
9          leds : out std_logic_vector(7 downto 0)
10         );
11 end entity;
12
13 architecture addressing_tb_a of addressing_tb is
14     signal async_reset : std_logic := '1';
15     signal slave_we : std_logic := '0';
16     signal slave_d : std_logic := '0';
17     signal slave_addr : std_logic_vector(2 downto 0) := "000";
18     signal clk_12MHz_sim : std_logic := '0';
19     signal clk_50MHz_sim : std_logic := '0';
20     signal counter : unsigned(7 downto 0) := (others => '0');
21 begin
22
23     clk_12MHz_sim <= not clk_12MHz_sim after 41.666666 ns;
24     clk_50MHz_sim <= not clk_50MHz_sim after 10 ns;
25
26     process(clk_12MHz_sim) is
27         variable cnt : unsigned(7 downto 0) := (others => '0');
28     begin
29         if rising_edge(clk_12MHz_sim) then
30             if cnt < 255 then
31                 cnt := cnt + 1;
32                 counter <= cnt;
33                 async_reset <= '1';
34             else
35                 counter <= cnt;
36                 async_reset <= '0';
37             end if;
38         end if;
39     end process;
40
41     addr_master_instance : entity work.addr_master
42     generic map(
43         variable_delay => variable_delay
44     )
45     port map(
```

```

46     clk => clk_50MHz_sim,
47     rst => async_reset,
48     adr => slave_addr,
49     we => slave_we,
50     d => slave_d
51 );
52
53 addr_slave_instance : entity work.addr_slave
54 port map(
55     clk => clk_12MHz_sim,
56     we => slave_we,
57     d => slave_d,
58     rst => async_reset,
59     addr => slave_addr,
60     leds => leds
61 );
62
63
64 end architecture addressing_tb_a;

```

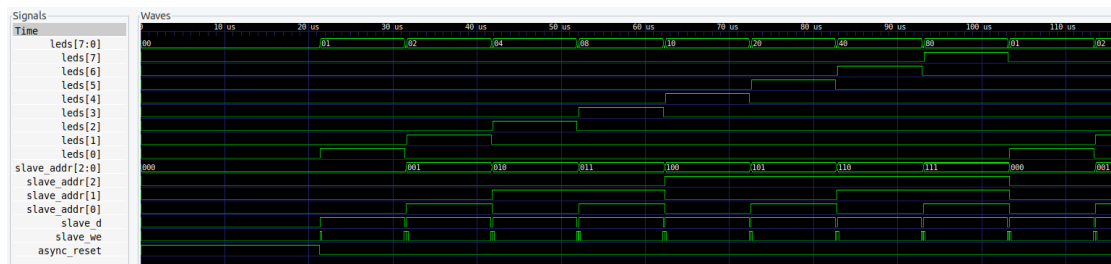


Abbildung 4: Simulation des Address-Master mit Slave

Die Simulation zeigt die korrekte Funktionalität des Masters und Slaves. Der Master durchläuft die Adressen von '000' bis '111', schaltet die entsprechenden LEDs ein und aus.

7 Metastabilität

Flip-Flops können metastabil werden, wenn asynchrone Eingangssignale sehr nahe an der aktiven Flanke des Taktes wechseln. In diesem Zustand bleibt das Flip-Flop für eine unbestimmte Zeit in einem undefinierten Spannungsbereich, was zu Glitches, Fehlern oder falschen Datenweiterleitungen führen kann.

7.1 Ursachen

- asynchrone Signaländerungen nahe am Taktflankenzeitpunkt
- nicht eingehaltene Setup- und Hold-Zeiten
- fehlende Synchronisation zwischen verschiedenen Taktdomänen

7.2 Auswirkungen

- fehlerhafte Signalausgabe
- unvorhersehbares Verhalten von State Machines
- sporadische Systemausfälle

7.3 Vermeidung

- Synchronisationsstufen (z. B. zwei Flip-Flops hintereinander)
- ausreichende Setup- und Hold-Zeiten
- Verwendung von Taktdomänen-Grenzen und Cross-Clock-Handshake